UNIVERSIDAD PRIVADA FRANZ TAMAYO

SEDE EL ALTO

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

ASIGNATURA: TESTING DE SOFTWARE



"PRUEBAS DE LOGIN VÍA API, SELENIUM Y CYPRESS"

INTEGRANTE: BRAYAN QUISPE MAMANI

DOCENTE: NELSON CRISTHIAN URUCHI QUISPE

EL ALTO - BOLIVIA

2026

Índice

- 1. Introducción y objetivo de la práctica
- 2. Fundamentación teórica
- 3. Preparación del entorno local
- 4. Punto 1: Procedimiento para ejecutar el login vía API
- 5. Punto 2: Procedimiento para ejecutar el login vía Selenium
- 6. Punto 3: Procedimiento para ejecutar el login vía Cypress
- 7. Resultados y evidencias (capturas)
- 8. Dificultades encontradas
- 9. Conclusiones
- 10. Anexos

1. Introducción y objetivo de la práctica

El presente informe documenta la ejecución de pruebas automatizadas sobre el inicio de sesión (login) del sistema de prueba appTesting. El sistema es una aplicación web desarrollada con Flask que se apoya en una base de datos MariaDB y que expone tanto una interfaz gráfica como una API REST. El objetivo de la práctica es levantar el sistema de forma local y verificar el login mediante tres herramientas distintas: una prueba de API, una prueba de interfaz con Selenium y una prueba de interfaz con Cypress.

Las tres pruebas se ejecutan con la misma cuenta de prueba, cuyas credenciales son el usuario pedrito@gmail.com y la contraseña bolivia123. De esta manera se comprueba que el mismo caso de prueba es verificable desde distintos niveles de la pirámide de automatización: la capa de servicios (API), y la capa de interfaz de usuario (end-to-end).

Al final del informe se presentan las capturas que demuestran el funcionamiento de cada prueba, tal como lo solicita la instrucción de la práctica.

2. Fundamentación teórica

Antes de describir los procedimientos conviene aclarar los conceptos y las herramientas utilizadas en la práctica.

2.1 API REST y el endpoint de sesión

Una API REST es un estilo de arquitectura para servicios web que expone sus operaciones a través del protocolo HTTP, usando los métodos GET, POST, PUT y DELETE sobre recursos identificados por una URL, y cuyo cuerpo se intercambia en formato JSON. El sistema appTesting define el recurso de sesión en la ruta /api/sesion. El login por API consiste en enviar una petición POST con el correo y la contraseña; el servidor busca al usuario, verifica la contraseña contra su hash almacenado y, si todo es correcto, responde con código 200 y los datos públicos del usuario.

2.2 Selenium

Selenium es una familia de herramientas para automatizar navegadores web. La librería selenium-webdriver permite que un programa Python controle un navegador real: abrir una página, localizar elementos por su identificador, escribir texto en los campos, hacer clic en botones y esperar a que aparezcan ciertas condiciones. La comunicación entre el código y el navegador se realiza mediante un componente intermedio llamado chromedriver.

2.3 Cypress

Cypress es un framework de pruebas de extremo a extremo escrito en JavaScript que se ejecuta dentro del mismo navegador donde vive la aplicación. Su API encadenable, con métodos como cy.visit, cy.get y cy.type, permite describir el recorrido del usuario de forma legible. Al igual que Selenium, comprueba el resultado mediante aserciones, pero con la ventaja de que espera de forma automática a que los elementos estén disponibles.

2.4 Flask, MariaDB y pytest

Flask es el microframework web con el que está construido el sistema: sirve la interfaz HTML y expone la API REST. MariaDB es el motor de base de datos relacional donde se guardan los usuarios, los productos y las ventas. pytest es la herramienta de pruebas de Python que descubre y ejecuta los archivos de prueba, y que en este caso también se usa para las pruebas de API y de Selenium.

3. Preparación del entorno local

Antes de ejecutar cualquier prueba se debe descargar el sistema y dejarlo funcionando de forma local, junto con su base de datos.

3.1 Requisitos

- Sistema operativo Linux (en esta práctica se usó EndeavourOS).
- Python 3 y Node.js con npm instalados.
- Servidor MariaDB instalado y en ejecución.

- Un navegador basado en Chromium y su chromedriver para Selenium.

3.2 Descarga del repositorio

Se clona el repositorio indicado en la instrucción:

```
git clone https://github.com/CrisUruchi/appTesting
cd appTesting
```

3.3 Base de datos

Se habilita e inicia el servidor MariaDB y luego se ejecuta el script setup_db.sh, que recrea la base de datos crud_productos, habilita la conexión de root sin contraseña (igual que db.py) y crea las tablas usuarios, productos, ventas y compradores aplicando las migraciones del repositorio.

```
sudo systemctl enable --now mariadb
bash setup_db.sh
```

3.4 Entorno de Python y dependencias

Se crea un entorno virtual y se instalan las dependencias declaradas en requirements.txt (Flask, mysql-connector-python, pytest, requests y selenium).

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install webdriver-manager   # opcional, ayuda con el driver
```

3.5 Cuenta de prueba

Se crea una cuenta con perfil admin (gerente) para poder iniciar sesión. El repositorio incluye el asistente crear_usuario.py, que pide nombre, correo, perfil y contraseña, y guarda la contraseña como hash. En esta práctica la cuenta creada fue pedrito@gmail.com con la contraseña bolivia123.

```
python crear_usuario.py
Nombre completo: Pedrito
Correo: pedrito@gmail.com
Perfil (admin/compras/vendedor): admin
Contrasena (minimo 8 caracteres): bolivia123
```

3.6 Inicio del servidor

Finalmente se inicia la aplicación. Flask queda escuchando en http://localhost:5000, que es la dirección que usan todas las pruebas.

```
python app.py
# * Running on http://127.0.0.1:5000
```

4. Punto 1: Procedimiento para ejecutar el login vía API

La prueba de login por API verifica el endpoint POST /api/sesion sin abrir ningún navegador, únicamente con peticiones HTTP.

4.1 Descripción del endpoint

El endpoint recibe un cuerpo JSON con las claves correo y contrasena. Si las credenciales son válidas, responde con código 200 y un objeto con los datos públicos del usuario (nombre, correo y rol). Si el usuario no existe, está inactivo o la contraseña no coincide, responde con código 401. El mismo recurso admite GET para consultar la sesión activa y DELETE para cerrarla.

4.2 Código de la prueba

El archivo 4_test_api_productos.py define una fixture de sesión que inicia el login una sola vez y conserva la cookie, y luego ejecuta quince casos de prueba sobre los productos. El fragmento del login es el siguiente:

Fragmento: login por API (4_test_api_productos.py)

```
def iniciar_sesion_api():
    correo = os.environ.get("API_EMAIL")
    contrasena = os.environ.get("API_PASSWORD")
    if not correo or not contrasena:
        pytest.skip("Configura API_EMAIL y API_PASSWORD")
    respuesta = requests.post(
        f"{BASE_URL}/api/sesion",
        json={"correo": correo, "contrasena": contrasena},
        headers=HEADERS,
    )
    assert respuesta.status_code == 200
```

Además, se usó un pequeño script, api_login_demo.py, que hace el POST y luego un GET a /api/sesion para mostrar la sesión activa de forma legible:

Fragmento: api_login_demo.py

```
respuesta = cliente.post(
    f"{BASE}/api/sesion",
    json={"correo": CORREO, "contrasena": CONTRASENA},
)
print(respuesta.status_code)
print(respuesta.json())
```

4.3 Procedimiento paso a paso

- Paso 1. Verificar que la aplicación esté corriendo en http://localhost:5000.

- Paso 2. Exportar las credenciales de la cuenta en variables de entorno.
- Paso 3. Ejecutar la prueba de API con pytest en modo detallado.
- Paso 4. Ejecutar el script api_login_demo.py para ver la respuesta del login.
- Paso 5. Observar que el estado devuelto es 200 y el perfil del usuario.

```
export API_EMAIL=pedrito@gmail.com
export API_PASSWORD=bolivia123
python -m pytest 4_test_api_productos.py -v
python api_login_demo.py
```

4.4 Resultado esperado

La prueba de API finaliza con quince casos aprobados y la sesión del usuario pedrito@gmail.com se inicia correctamente con perfil Gerente. La evidencia se muestra en la Figura 1.

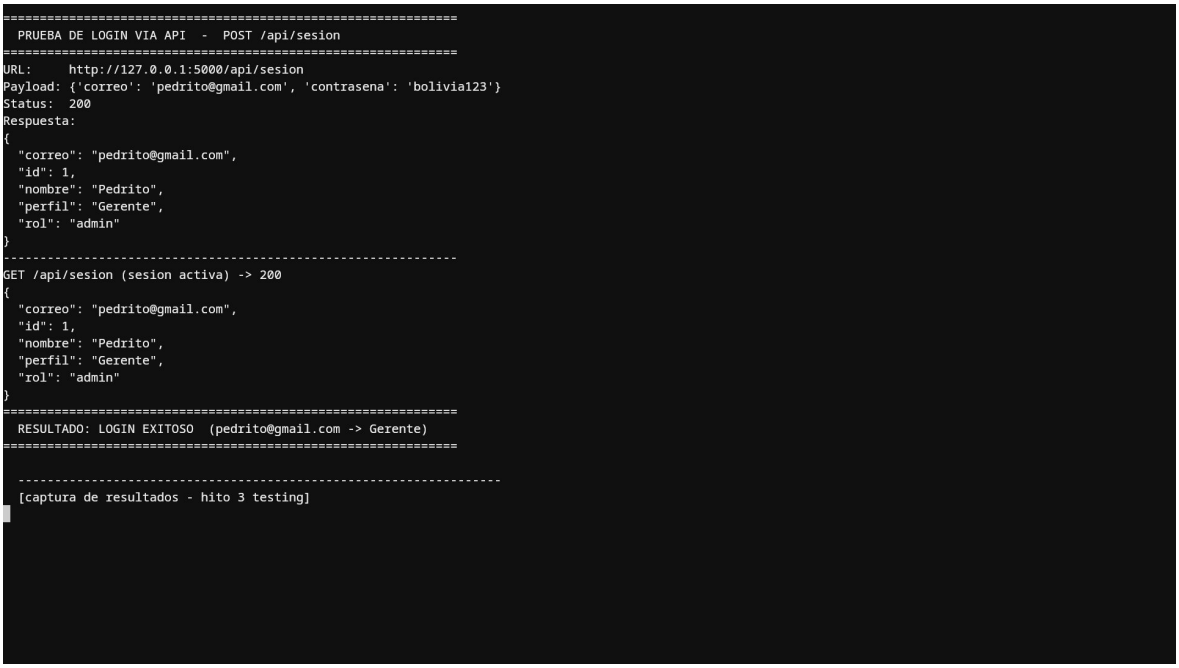


Figura 1. Ejecución del login vía API: POST /api/sesion responde 200 y muestra la sesión activa de pedrito@gmail.com con perfil Gerente.

5. Punto 2: Procedimiento para ejecutar el login vía Selenium

La prueba con Selenium automatiza un navegador real para simular el ingreso del usuario a través de la interfaz gráfica.

5.1 Descripción de la prueba

El archivo 7_pruebaUISelenium.py abre el navegador, navega a la página principal, localiza los campos del formulario de login por su identificador HTML, escribe el correo y la contraseña, envía el formulario y espera a que aparezca la vista de la aplicación. Después, como verificación adicional, registra un producto y comprueba que aparezca en la tabla.

Los elementos de la interfaz que se utilizan son: el contenedor del formulario #vista-login, el campo #correo-login, el campo #contrasena-login, el formulario #form-login, y ya dentro de la aplicación, #vista-app y #usuario-nombre.

5.2 Configuración del navegador

Para que Selenium no tenga que descargar el controlador en tiempo de ejecución, el archivo conftest.py fija la ubicación del navegador y del chromedriver, y garantiza que la variable DISPLAY esté definida en entornos con servidor gráfico:

Fragmento: conftest.py

```
CHROME_BIN = "/usr/bin/brave-origin-nightly"
DRIVER_BIN = os.path.expanduser(
    "~/.cache/selenium/chromedriver/linux64/"
    "155.0.8059.39/chromedriver")
opts = Options()
opts.binary_location = CHROME_BIN
svc = Service(executable_path=DRIVER_BIN)
```

Las credenciales se leen del archivo usuarios_prueba.json, que contiene la cuenta con perfil admin:

Fragmento: usuarios_prueba.json

```
{
  "usuarios": [
    {
      "nombre": "Pedrito",
      "correo": "pedrito@gmail.com",
      "contrasena": "bolivia123",
      "rol": "admin"
    }
  ]
}
```

5.3 Procedimiento paso a paso

- Paso 1. Confirmar que la aplicación Flask esté activa en el puerto 5000.
- Paso 2. Verificar que el entorno virtual tenga instalada la librería selenium.

- Paso 3. Comprobar la configuración del navegador y del chromedriver (conftest.py).
- Paso 4. Crear el archivo usuarios_prueba.json con la cuenta admin.
- Paso 5. Ejecutar la prueba con pytest y observar cómo se abre el navegador.

```
source .venv/bin/activate
python -m pytest 7_pruebaUISelenium.py -v -s
```

5.4 Resultado esperado

El navegador se abre, se escriben las credenciales en el formulario y se envía. La Figura 2 muestra el formulario de inicio de sesión con los datos ya ingresados, y la Figura 3 muestra la aplicación autenticada, con el nombre del usuario en la barra superior. La prueba finaliza con un caso aprobado.

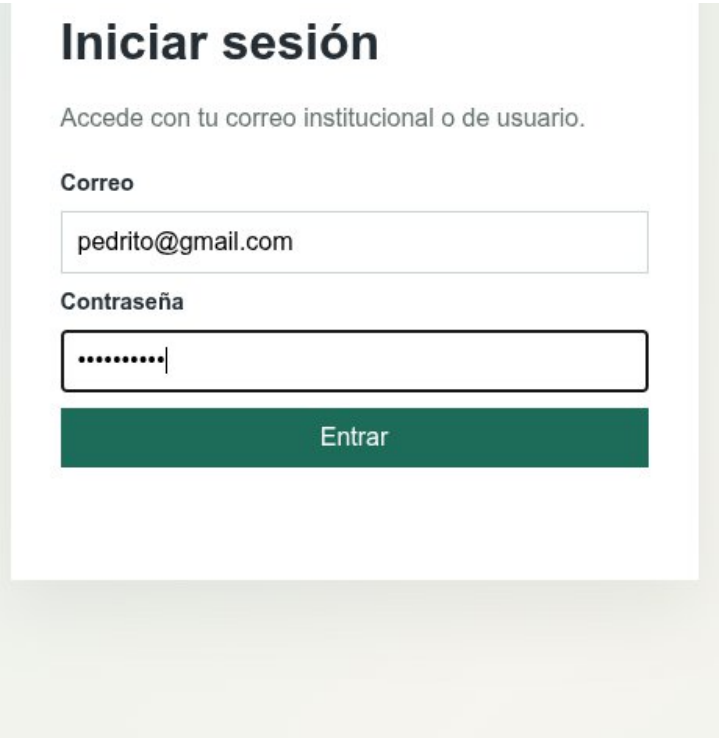


Figura 2. Formulario de inicio de sesión diligenciado por Selenium con el correo pedrito@gmail.com y la contraseña de prueba.

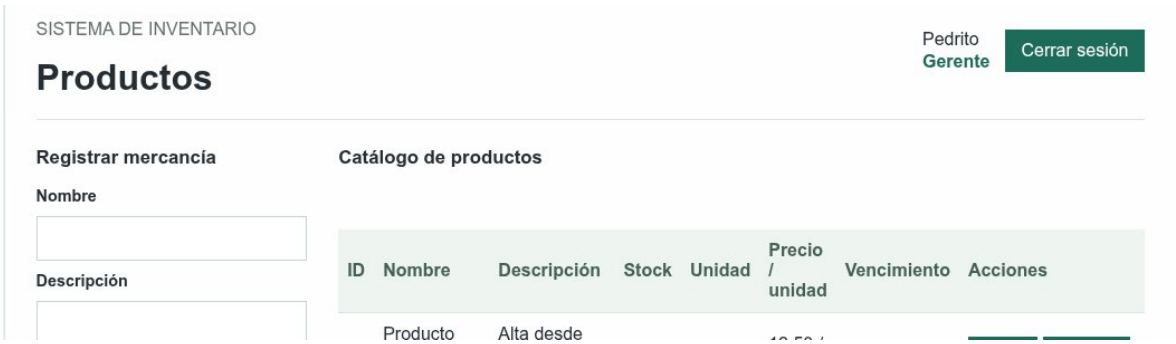


Figura 3. Aplicación con la sesión iniciada tras enviar el formulario: se observa el nombre del usuario en la barra superior.

6. Punto 3: Procedimiento para ejecutar el login vía Cypress

La prueba con Cypress verifica el mismo login exitoso, pero desde un framework de pruebas de extremo a extremo escrito en JavaScript.

6.1 Descripción de la prueba

El archivo 8_pruebaUICypress.cy.js visita la raíz del sitio, comprueba que la vista de login sea visible, escribe el correo y la contraseña y envía el formulario. Luego verifica que la vista de la aplicación y los datos del usuario sean visibles y no estén vacíos.

Fragmento: 8_pruebaUICypress.cy.js

```
cy.visit("/");
cy.get("#vista-login").should("be.visible");
cy.get("#correo-login").type("pedrito@gmail.com");
cy.get("#contrasena-login").type("bolivia123",
                                 { log: false });
cy.get("#form-login").submit();
cy.get("#vista-app").should("be.visible");
cy.get("#usuario-nombre").should("be.visible")
                         .and("not.be.empty");
```

6.2 Configuración de Cypress

El archivo cypress.config.js define la dirección base de la aplicación y lee las credenciales del archivo usuarios_prueba.json dentro de la función setupNodeEvents, de modo que la prueba no tenga las credenciales escritas en el propio archivo de prueba:

Fragmento: cypress.config.js

```
module.exports = defineConfig({
  e2e: {
    baseUrl: "http://localhost:5000",
    specPattern: "8_pruebaUICypress.cy.js",
    setupNodeEvents(on, config) {
      config.env = { ...config.env, ...credenciales };
      return config;
    },
  },
});
```

6.3 Procedimiento paso a paso

- Paso 1. Instalar Node.js y las dependencias del proyecto con npm install.
- Paso 2. Confirmar que la aplicación Flask esté activa en el puerto 5000.
- Paso 3. Verificar el archivo de soporte cypress/support/e2e.js.
- Paso 4. Ejecutar la prueba en modo consola indicando el archivo de prueba.
- Paso 5. Observar el resumen con el resultado de la ejecución.

```
npm install
npx cypress run --spec 8_pruebaUICypress.cy.js
# o simplemente:
npm run test:cypress
# modo interactivo (opcional):
npx cypress open
```

6.4 Resultado esperado

Cypress abre un navegador controlado, ejecuta el caso y muestra el resumen con todos los pasos aprobados. La consola informa que la prueba pasó (All specs passed). La evidencia se muestra en la Figura 4.

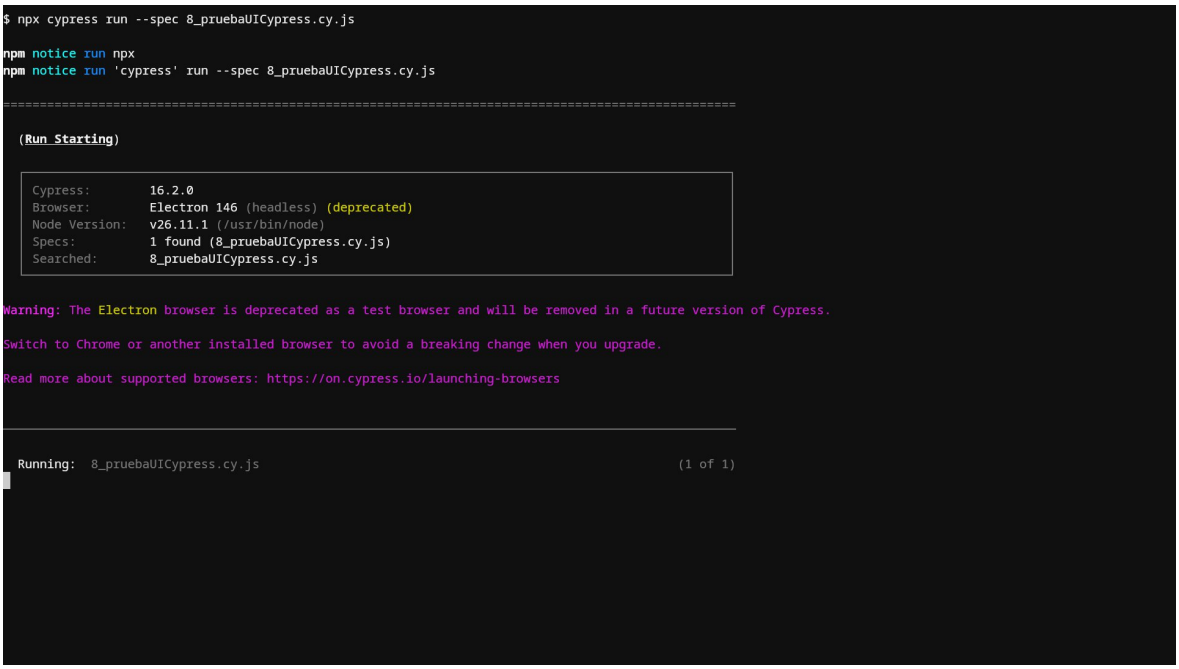


Figura 4. Ejecución del login vía Cypress: la prueba del caso de inicio de sesión finaliza aprobada.

7. Resultados y evidencias (capturas)

En esta sección se resumen los resultados obtenidos en cada una de las pruebas ejecutadas. Todas las ejecuciones se realizaron contra la aplicación local en http://localhost:5000 con la cuenta pedrito@gmail.com.

Tabla 1. Resumen de resultados de las pruebas ejecutadas.

Prueba	Herramienta	Comando	Resultado
Login y productos (API)	pytest + requests	pytest 4_test_api_productos.py	15 aprobadas
Autenticación (unitarias)	pytest	pytest 5_test_autenticacion.py	13 aprobadas
Login (interfaz)	Selenium	pytest 7_pruebaUISelenium.py	1 aprobada
Login (interfaz)	Cypress	cypress run --spec 8_...cy.js	1 aprobada

Como demostración del funcionamiento, las Figuras 1 a 4 presentan, de forma ordenada, la ejecución del login por API (Figura 1), el formulario de inicio de sesión diligenciado con Selenium (Figura 2), la aplicación autenticada tras el login con Selenium (Figura 3) y la ejecución de la prueba con Cypress (Figura 4).

8. Dificultades encontradas

La principal dificultad no estuvo en el código de las pruebas, sino en la preparación del entorno. Selenium intentaba descargar automáticamente su controlador a través de Selenium Manager y la descarga se quedaba detenida; se resolvió indicando explícitamente el navegador y el chromedriver ya disponibles en el equipo mediante el archivo conftest.py.

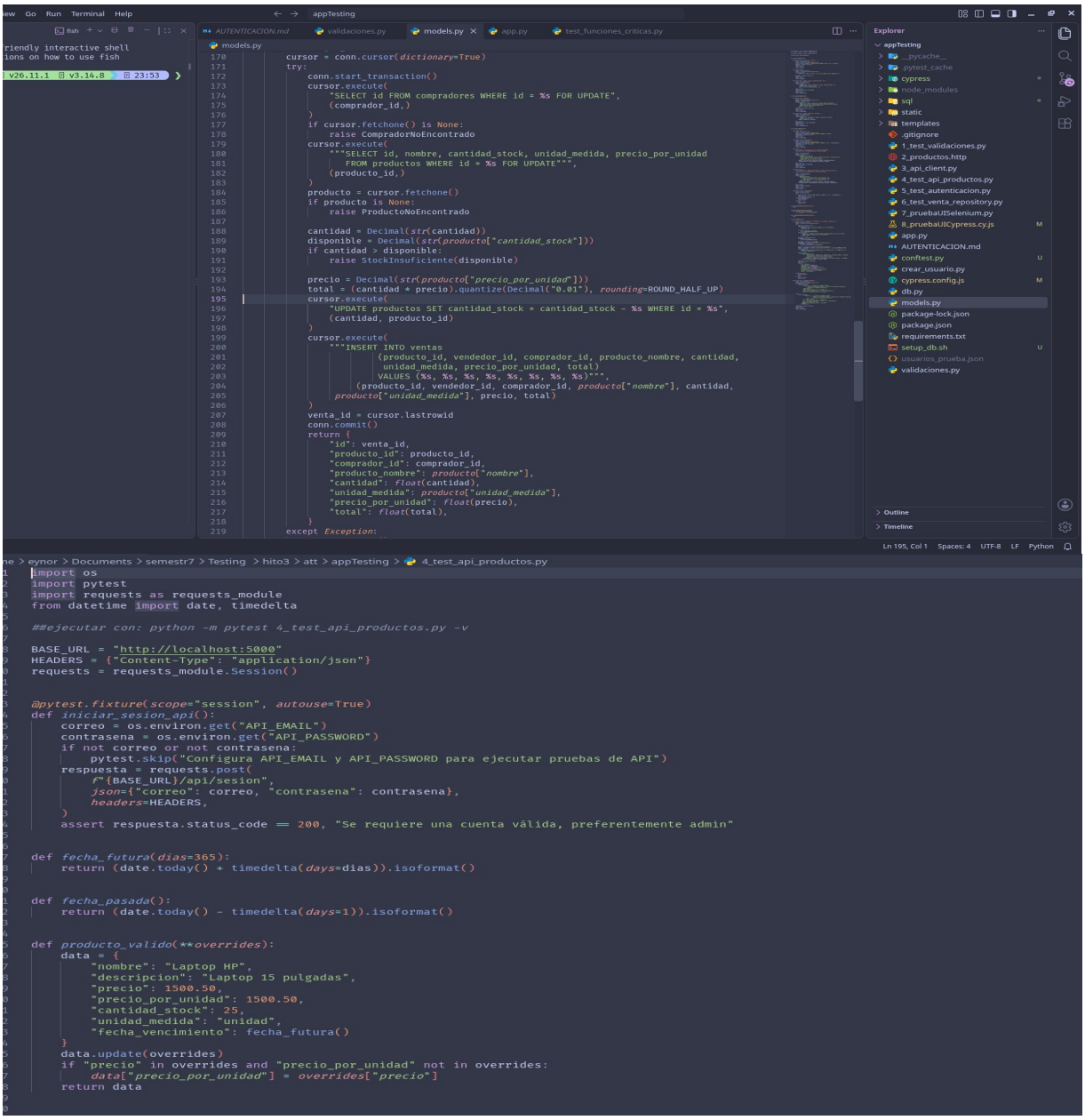
En Cypress, la versión fijada en package.json utilizaba una versión de Electron que se cerraba inesperadamente al iniciar; se actualizó Cypress a una versión compatible. Además, la versión nueva dejó de admitir la lectura de variables con Cypress.env en el archivo de prueba, por lo que las credenciales se colocaron de forma directa en el archivo de la prueba.

9. Conclusiones

Se logró levantar el sistema de forma local y ejecutar con éxito el login desde los tres enfoques solicitados. La prueba de API confirmó el funcionamiento del endpoint /api/sesion; la prueba de Selenium confirmó el ingreso a través de la interfaz gráfica con un navegador real; y la prueba de Cypress confirmó el mismo recorrido desde un framework de pruebas de extremo a extremo.

La práctica permitió comprobar que un mismo caso de prueba puede verificarse en distintos niveles de automatización y que la mayor parte del esfuerzo se concentra en dejar el entorno reproducible. Documentar cada paso, desde la base de datos hasta la ejecución de las pruebas, facilita repetir el procedimiento y garantiza que los resultados sean verificables.

10. Anexos



```
cursor = conn.cursor(dictionary=True)
try:
    conn.start_transaction()
    cursor.execute(
        "SELECT id FROM compradores WHERE id = %s FOR UPDATE",
        (comprador_id,)
    )
    if cursor.fetchone() is None:
        raise CompradorNoEncontrado
    cursor.execute(
        """SELECT id, nombre, cantidad_stock, unidad_medida, precio_por_unidad
           FROM productos WHERE id = %s FOR UPDATE""",
        (producto_id,)
    )
    producto = cursor.fetchone()
    if producto is None:
        raise ProductoNoEncontrado

    cantidad = Decimal(str(cantidad))
    disponible = Decimal(str(producto["cantidad_stock"]))
    if cantidad > disponible:
        raise StockInsuficiente(disponible)

    precio = Decimal(str(producto["precio_por_unidad"]))
    total = (cantidad * precio).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    cursor.execute(
        "UPDATE productos SET cantidad_stock = cantidad_stock - %s WHERE id = %s",
        (cantidad, producto_id)
    )
    cursor.execute(
        """INSERT INTO ventas
                  (producto_id, vendedor_id, comprador_id, producto_nombre, cantidad,
                   unidad_medida, precio_por_unidad, total)
                  VALUES (%s, %s, %s, %s, %s, %s, %s, %s)""",
               (producto_id, vendedor_id, comprador_id, producto["nombre"], cantidad,
            producto["unidad_medida"], precio, total)
    )
    venta_id = cursor.lastrowid
    conn.commit()
    return {
        "id": venta_id,
        "producto_id": producto_id,
        "comprador_id": comprador_id,
        "producto_nombre": producto["nombre"],
        "cantidad": float(cantidad),
        "unidad_medida": producto["unidad_medida"],
        "precio_por_unidad": float(precio),
        "total": float(total),
    }
except Exception:
```

```
import os
import pytest
import requests as requests_module
from datetime import date, timedelta

##ejecutar con: python -m pytest 4_test_api_productos.py -v

BASE_URL = "http://localhost:5000"
HEADERS = {"Content-Type": "application/json"}
requests = requests_module.Session()


@pytest.fixture(scope="session", autouse=True)
def iniciar_sesion_api():
    correo = os.environ.get("API_EMAIL")
    contrasena = os.environ.get("API_PASSWORD")
    if not correo or not contrasena:
        pytest.skip("Configura API_EMAIL y API_PASSWORD para ejecutar pruebas de API")
    respuesta = requests.post(
        f"{BASE_URL}/api/sesion",
        json={"correo": correo, "contrasena": contrasena},
        headers=HEADERS,
    )
    assert respuesta.status_code == 200, "Se requiere una cuenta válida, preferentemente admin"


def fecha_futura(dias=365):
    return (date.today() + timedelta(days=dias)).isoformat()


def fecha_pasada():
    return (date.today() - timedelta(days=1)).isoformat()


def producto_valido(**overrides):
    data = {
        "nombre": "Laptop HP",
        "descripcion": "Laptop 15 pulgadas",
        "precio": 1500.50,
        "precio_por_unidad": 1500.50,
        "cantidad_stock": 25,
        "unidad_medida": "unidad",
        "fecha_vencimiento": fecha_futura()
    }
    data.update(overrides)
    if "precio" in overrides and "precio_por_unidad" not in overrides:
        data["precio_por_unidad"] = overrides["precio"]
    return data
```

```python

# ═══════════════════════════════════════════════════════
# Fixture: crea un producto y lo elimina al final
# ═══════════════════════════════════════════════════════
@pytest.fixture
def producto_creado():
    r = requests.post(f"{BASE_URL}/api/productos",
                      json=producto_valido(), headers=HEADERS)
    assert r.status_code == 201
    producto = r.json()
    yield producto
    # Cleanup
    requests.delete(f"{BASE_URL}/api/productos/{producto['id']}", headers=HEADERS)


# ═══════════════════════════════════════════════════════
# 01 - Listar productos
# ═══════════════════════════════════════════════════════
def test_listar_productos_devuelve_200():
    r = requests.get(f"{BASE_URL}/api/productos", headers=HEADERS)
    assert r.status_code == 200
    assert isinstance(r.json(), list)


# ═══════════════════════════════════════════════════════
# 02 - Crear producto válido
# ═══════════════════════════════════════════════════════
def test_crear_producto_valido_devuelve_201():
    payload = producto_valido()
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 201
    body = r.json()
    assert body["id"] is not None
    assert body["nombre"] == payload["nombre"]
    assert body["cantidad_stock"] == payload["cantidad_stock"]
    assert body["unidad_medida"] == payload["unidad_medida"]
    assert body["precio_por_unidad"] == payload["precio_por_unidad"]
    # Cleanup
    requests.delete(f"{BASE_URL}/api/productos/{body['id']}", headers=HEADERS)


# ═══════════════════════════════════════════════════════
# 03 - Obtener producto por id
# ═══════════════════════════════════════════════════════
def test_obtener_producto_existente(producto_creado):
    r = requests.get(f"{BASE_URL}/api/productos/{producto_creado['id']}",
                     headers=HEADERS)
    assert r.status_code == 200
```

```python
# ═══════════════════════════════════════════════════════════
# 04 - Obtener producto inexistente
# ═══════════════════════════════════════════════════════════
def test_obtener_producto_inexistente_devuelve_404():
    r = requests.get(f"{BASE_URL}/api/productos/999999", headers=HEADERS)
    assert r.status_code == 404
    assert "error" in r.json()


# ═══════════════════════════════════════════════════════════
# 05 - Crear con precio negativo
# ═══════════════════════════════════════════════════════════
def test_crear_con_precio_negativo_devuelve_400():
    payload = producto_valido(precio=-100)
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400
    assert "precio" in r.json()["error"].lower()


# ═══════════════════════════════════════════════════════════
# 06 - Crear con precio cero
# ═══════════════════════════════════════════════════════════
def test_crear_con_precio_cero_devuelve_400():
    payload = producto_valido(precio=0)
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400


# ═══════════════════════════════════════════════════════════
# 07 - Crear con precio no numérico
# ═══════════════════════════════════════════════════════════
def test_crear_con_precio_no_numerico_devuelve_400():
    payload = producto_valido(precio="abc")
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400


# ═══════════════════════════════════════════════════════════
# 08 - Crear con nombre vacío
# ═══════════════════════════════════════════════════════════
def test_crear_con_nombre_vacio_devuelve_400():
    payload = producto_valido(nombre="")
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400


# ═══════════════════════════════════════════════════════════
# 09 - Crear con nombre null
# ═══════════════════════════════════════════════════════════
```

```python
# ═══════════════════════════════════════════════════════
# 09 - Crear con nombre null
# ═══════════════════════════════════════════════════════
def test_crear_con_nombre_null_devuelve_400():
    payload = producto_valido(nombre=None)
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400


# ═══════════════════════════════════════════════════════
# 10 - Crear con fecha pasada
# ═══════════════════════════════════════════════════════
def test_crear_con_fecha_pasada_devuelve_400():
    payload = producto_valido(fecha_vencimiento=fecha_pasada())
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400
    assert "fecha" in r.json()["error"].lower()


# ═══════════════════════════════════════════════════════
# 11 - Crear con fecha formato inválido
# ═══════════════════════════════════════════════════════
def test_crear_con_fecha_invalida_devuelve_400():
    payload = producto_valido(fecha_vencimiento="31-12-2026")
    r = requests.post(f"{BASE_URL}/api/productos", json=payload, headers=HEADERS)
    assert r.status_code == 400


# ═══════════════════════════════════════════════════════
# 12 - Actualizar producto
# ═══════════════════════════════════════════════════════
def test_actualizar_producto(producto_creado):
    actualizado = producto_valido(nombre="Laptop Actualizada", precio=2000)
    r = requests.put(f"{BASE_URL}/api/productos/{producto_creado['id']}",
                     json=actualizado, headers=HEADERS)
    assert r.status_code == 200
    # Verificar que se guardó
    r2 = requests.get(f"{BASE_URL}/api/productos/{producto_creado['id']}",
                      headers=HEADERS)
    assert r2.json()["nombre"] == "Laptop Actualizada"
    assert r2.json()["precio"] == 2000.0


# ═══════════════════════════════════════════════════════
# 13 - Actualizar producto inexistente
# ═══════════════════════════════════════════════════════
def test_actualizar_producto_inexistente_devuelve_404():
    actualizado = producto_valido()
    r = requests.put(f"{BASE_URL}/api/productos/999999",
                     json=actualizado, headers=HEADERS)
    assert r.status_code == 404
```


```python
# ═══════════════════════════════════════════════════════
# 13 - Actualizar producto inexistente
# ═══════════════════════════════════════════════════════
def test_actualizar_producto_inexistente_devuelve_404():
    actualizado = producto_valido()
    r = requests.put(f"{BASE_URL}/api/productos/999999",
                     json=actualizado, headers=HEADERS)
    assert r.status_code == 404
```


```python


# ═══════════════════════════════════════════════════════
# 14 - Eliminar producto
# ═══════════════════════════════════════════════════════
def test_eliminar_producto(producto_creado):
    r = requests.delete(f"{BASE_URL}/api/productos/{producto_creado['id']}",
                        headers=HEADERS)
    assert r.status_code == 200
    # Verificar que ya no existe
    r2 = requests.get(f"{BASE_URL}/api/productos/{producto_creado['id']}",
                      headers=HEADERS)
    assert r2.status_code == 404


# ═══════════════════════════════════════════════════════
# 15 - Eliminar producto inexistente
# ═══════════════════════════════════════════════════════
def test_eliminar_producto_inexistente_devuelve_404():
    r = requests.delete(f"{BASE_URL}/api/productos/999999", headers=HEADERS)
    assert r.status_code == 404
```